

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Харківський національний університет імені В.Н. Каразіна

Навчально-науковий інститут комп'ютерних наук та штучного інтелекту

Кафедра математичного моделювання та аналізу даних

ЛАБОЛАТОРНА РОБОТА № 3

на тему: «Створення запитів у MS SQL»

Виконав: студент 3 курсу групи кс31
Спеціальності 124 «Комп'ютерні
науки»

Кірієнко Данило

Прийняв: к.т.н., доцент каф. ММ та
АД

Коробчинський К.П.

Мета роботи:

Навчитися виконувати багатотабличні запити, використовувати каскадні дії для забезпечення цілісності даних, застосовувати сортування результатів запитів, використовувати булеві та реляційні оператори для фільтрації даних, а також освоїти різні види JOIN для об'єднання таблиць та аналізувати продуктивність SQL-запитів.

Завдання 1.

Пройти тестування та додати скріншот з оцінкою у звіт.

Завдання 2.

За своїм варіантом (основного, по якому були виконані практичні роботи 3, 4, 5 та 6) додати опис предметної області (скопювати з варіанту), логічну і фізичну моделі (скопювати з практичної роботи №5).

Завдання 3.

Використати розгорнутий інстанс MS SQL на Docker, та додати базу даних використовуючи скрипти SETUP.SQL та INSERT.SQL.

Завдання 4. Формування запитів із використанням багатотабличних дій

Багатотабличні запити дозволяють отримувати дані з кількох таблиць одночасно. Це досягається за допомогою підзапитів, операторів JOIN та об'єднання результатів (UNION). Такий підхід забезпечує гнучкість у вибірці даних та дає змогу отримати потрібну інформацію без дублювання.

- Скласти декілька SQL-запитів із використанням багатотабличних дій.
- До звіту додати скріншоти виконання запитів та їхніх результатів.
- Зберегти код запитів у файл QUERY.SQL та оновити його на своїй сторінці на GitHub.

Завдання 5. Формування запитів із каскадними діями

Каскадні дії (CASCADE) використовуються для автоматизації оновлення або видалення пов'язаних записів у таблицях. Вони допомагають зберегти узгодженість даних та мінімізувати помилки, що можуть виникнути через зміну значень у зовнішніх ключах.

- Створити таблиці з обмеженнями FOREIGN KEY та опціями ON DELETE CASCADE та ON UPDATE CASCADE.
- Скласти декілька SQL-запитів для перевірки каскадних оновлень та видалень.
- Додати до звіту скріншоти виконання запитів.
- Код запитів зберегти у файл QUERY.SQL та оновити його на GitHub.

Завдання 6. Формування запитів із використанням сортування даних (ORDER BY)

Оператор ORDER BY використовується для впорядкування результатів запиту за зростанням або спаданням. Це дозволяє ефективно аналізувати інформацію та знаходити необхідні записи.

- Написати декілька SQL-запитів із використанням ORDER BY для впорядкування даних за різними критеріями.
- Виконати запити та зробити скріншоти результатів.
- Додати до звіту отримані результати.
- Зберегти код у QUERY.SQL та оновити GitHub-репозиторій.

Завдання 7. Використання булевих та реляційних операторів

Булеві (AND, OR, NOT) та реляційні оператори (=, >, <, >=, <=, <>) дозволяють створювати складні умови у запитах, що підвищує точність вибірки даних.

- Написати декілька SQL-запитів із використанням булевих та реляційних операторів.
- Продемонструвати роботу кожного запиту та зробити відповідні скріншоти.

- Додати результати виконання запитів до звіту.
- Код запитів зберегти у QUERY.SQL та оновити його на GitHub.

Завдання 8. Розширене використання оператора JOIN у складних запитах

Окрім стандартного INNER JOIN, застосовуються й інші види об'єднання таблиць: LEFT JOIN, RIGHT JOIN, FULL JOIN, які дозволяють включати у вибірку неповні збіги між таблицями. Це забезпечує ширші можливості для аналізу даних та їх коректної інтерпретації. Додати декілька запитів за тематикою Вашого варіанта. До звіту додати скріншот успішного додавання, а у файл QUERY.SQL відповідний код та оновити на свою сторінку на github.

Завдання 9. Застосування оператора GROUP BY та умови HAVING у поєднанні з JOIN

Оператор GROUP BY використовується для групування записів за визначеними атрибутами, а умова HAVING дозволяє застосовувати фільтрацію до групованих даних. У комбінації з JOIN це дає змогу аналізувати дані за категоріями та отримувати статистичні підсумки. Додати 5 запитів за тематикою Вашого варіанта. До звіту додати скріншот успішного додавання, а у файл QUERY.SQL відповідний код та оновити на свою сторінку на github.

Завдання 10. Аналіз продуктивності запитів

Для аналізу продуктивності запитів у MS SQL використайте EXPLAIN (Execution Plan). Додайте скріншоти з поясненням та аналізом попередніх запитів.

Завдання 11.

Завантажити файли до Git-репозиторію та додати посилання у звіт. Репозиторій повинен бути закритим, а доступ відкритий лише для викладачів практики, лабораторних робіт та лектора (k.korobchinskiy@karazin.ua).

Використовуючи мову T-SQL зробити скрипти.

- створення таблиць та змінення структури таблиць БД – SETUP.SQL;
- занесення інформації в таблиці БД – INSERT.SQL.
- зміни/оновлення/видалення інформації у таблицях БД – UPDATE.SQL;
- запитів – QUERY.SQL.

Всі файли за архівувати та прикипіти до STM.

Завдання 12.

Виконати запити за Вашим варіантом відповідно до опису предметної області основного варіанта.

Завдання 13.

Додати висновки. До оформити звіт та завантажити на STM.

Хід роботи:

Завдання 2.

Варіант №15

Ферма

На ферме имеется несколько участков земли, им присвоены уникальные номера. Каждый участок характеризуется площадью, наличием или отсутствием орошения, посеянной в текущем сезоне культурой. Известна средняя урожайность каждой из возделываемых культур, а также перечень внесенных на каждый участок в данном сезоне удобрений. Для каждой культуры известно: название, урожайность по участкам, нужно ли для нее орошение, какие нужны удобрения. Каждый участок обслуживает одна бригада, но любая бригада может обслуживать более одного участка. Бригада имеет уникальный номер и характеризуется: Ф.И.О. бригадира, Ф.И.О. работников, количеством единиц каждого вида сельхозтехники. О каждом работнике известно, какими видами сельхозмашин (одним, несколькими, ни одним) он может управлять. Работники трудятся на любом из участков бригады, пользуясь техникой, которая есть в бригаде.

Запросы

- ✓ Определить площадь, которая отведена на ферме под каждую культуру с указанием, на каком количестве участков посеяна каждая культура;
- ✓ Вывести всех рабочих заданной бригады с перечнем сельхозтехники, на которой они умеют работать, если такая техника в бригаде имеется, а также определить те виды техники, на которой могут работать члены бригады, но в бригаде такой техники нет, хотя она есть в других бригадах
- д л я участков данной бригады найти культуру с максимальной урожайностью и проверить, посеяна ли эта культура в текущем сезоне на том участке, где ее урожайность максимальна
- ✓ Найти бригады, которые располагают заданным видом техники в количестве, большем чем среднее количество этой техники по бригадам на всей ферме;
- ✓ Составить список участков (с фамилиями их бригадиров), на которых внесены удобрения, не соответствующие культуре, или не выполняются требования по орошению культуры.

Транзакции

- ✓ Купить новую сельхозтехнику
- ✓ Нанять/уволить сельхозработных (в том числе и бригадиров).

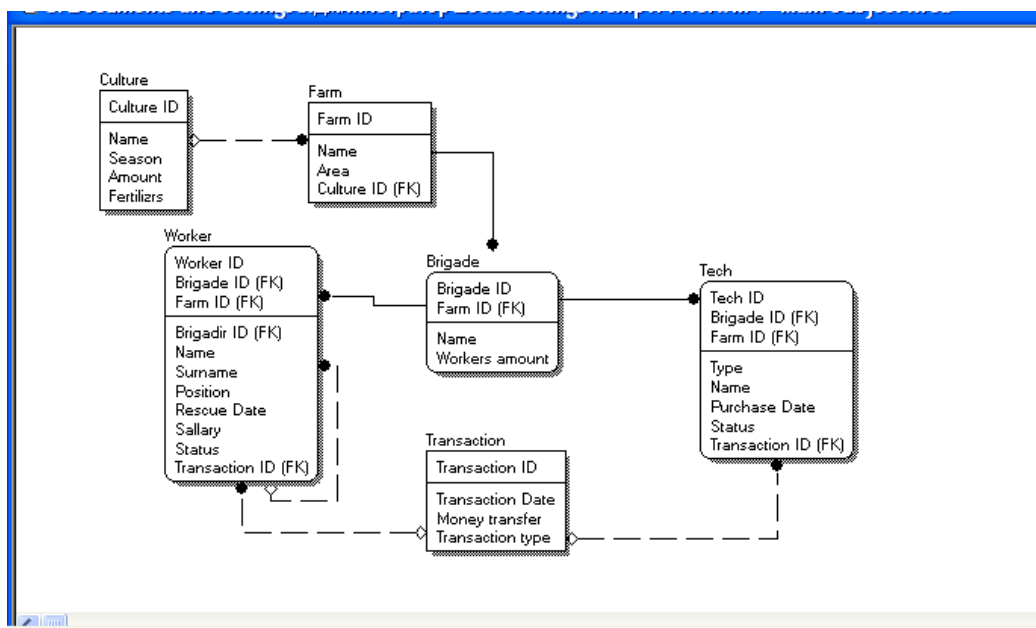


Рисунок 1 - Логічна модель

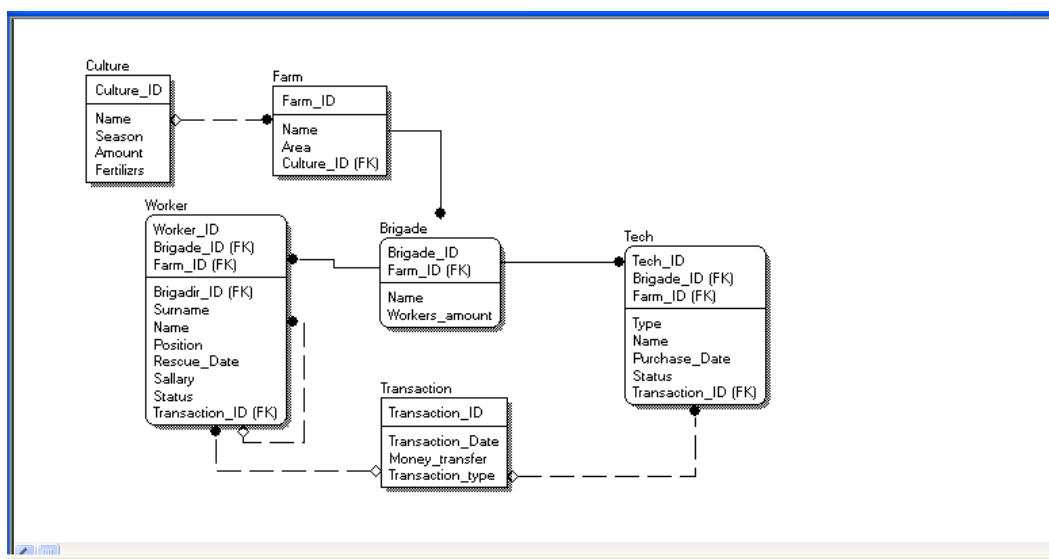


Рисунок 2 - Фізична модель

Завдання 3.

Оскільки до цього виконував роботи в MySQL, прийшлося переробити файли для MS SQL. Тому з цієї лабораторної роботи файли на гітхаб вже будуть сумісні з MS SQL. Також було змінено дані з української на англійську.

Завдання 4.

Приклади запитів із використанням багатотабличних дій:

```

SELECT
    w.Name          AS WorkerName,
    w.Surname       AS WorkerSurname,
    f.Name          AS FarmName,
    c.Name          AS CultureName
FROM Worker AS w
JOIN Brigade AS b ON w.Brigade_ID = b.Brigade_ID
                  AND w.Farm_ID    = b.Farm_ID
JOIN Farm AS f ON b.Farm_ID    = f.Farm_ID
JOIN Culture AS c ON f.Culture_ID = c.Culture_ID;
GO

SELECT
    c.Name          AS CultureName,
    AVG(f.Area)     AS AvgAreaByCulture
FROM Culture AS c
JOIN Farm AS f
    ON c.Culture_ID = f.Culture_ID
GROUP BY
    c.Culture_ID,
    c.Name
HAVING
    AVG(f.Area) > (
        SELECT AVG(Area)
        FROM Farm
    );

SELECT
    w.Name          AS EntityName,
    b.Name          AS BrigadeName,
    f.Name          AS FarmName,
    'Worker'       AS EntityType
FROM Worker AS w
JOIN Brigade AS b ON w.Brigade_ID = b.Brigade_ID
                  AND w.Farm_ID    = b.Farm_ID
JOIN Farm AS f ON b.Farm_ID    = f.Farm_ID
WHERE w.Status = 'Active'

UNION

SELECT
    t.Name          AS EntityName,
    b.Name          AS BrigadeName,
    f.Name          AS FarmName,
    'Tech'         AS EntityType
FROM Tech AS t

```

```

JOIN Brigade AS b ON t.Brigade_ID = b.Brigade_ID
                AND t.Farm_ID = b.Farm_ID
JOIN Farm AS f ON b.Farm_ID = f.Farm_ID
WHERE t.Status = 'Working';
GO

```

	WorkerName	WorkerSurname	FarmName	CultureName		CultureName	AvgAreaByCulture
1	Name1	Surname1	Farm 1	Barley	1	Wheat	150
2	Name2	Surname2	Farm 2	Com	2	Barley	143
3	Name3	Surname3	Farm 3	Wheat	3	Barley	151
4	Name4	Surname4	Farm 4	Wheat	4	Barley	173
5	Name5	Surname5	Farm 5	Potato	5	Barley	139
6	Name6	Surname6	Farm 6	Sunflower	6	Wheat	168
7	Name7	Surname7	Farm 7	Barley	7	Sunflower	190
8	Name8	Surname8	Farm 8	Barley	8	Barley	168

	EntityName	BrigadeName	FarmName	EntityType
1	Combine	Brigade 39	Farm 39	Tech
2	Combine	Brigade 47	Farm 47	Tech
3	Combine	Brigade 60	Farm 60	Tech
4	Combine	Brigade 61	Farm 61	Tech
5	Combine	Brigade 69	Farm 69	Tech
6	Combine	Brigade 84	Farm 84	Tech
7	Combine	Brigade 94	Farm 94	Tech
8	Name1	Brigade 1	Farm 1	Worker

Рисунок 3 - скріншоти результатів багатотабличних запитів

1. Цей запит використовує JOIN для об'єднання таблиць 'Worker', 'Brigade', 'Farm' та 'Culture' за відповідними ключами, щоб у результаті отримати ім'я працівника, прізвище, назву ферми та культуру в одному рядку.
2. Цей запит застосовує підзапит, який теж працює з двох таблиць (Culture і Farm): для кожної культури обчислює середню площу її ферм, а потім повертає ті культури, у яких ця середня площа більша за загальну середню площу по всіх фермах:
3. Цей запит використовує UNION, щоб об'єднати дві багатотабличні вибірки: список всіх працівників зі статусом Active (JOIN з Worker → Brigade → Farm) і список всієї техніки зі статусом Working (JOIN Tech → Brigade → Farm)

Завдання 5.

Спочатку створимо додаткові таблиці з обмеженнями FOREIGN KEY та опціями ON DELETE CASCADE та ON UPDATE CASCADE:

```
CREATE TABLE Department (  
    Dept_ID INT PRIMARY KEY,  
    Dept_Name VARCHAR(100)  
);  
GO  
  
CREATE TABLE Employee (  
    Emp_ID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Dept_ID INT,  
    FOREIGN KEY (Dept_ID) REFERENCES Department (Dept_ID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);  
GO  
  
CREATE TABLE Equipment (  
    Equip_ID INT PRIMARY KEY,  
    Emp_ID INT,  
    Equipment_Name VARCHAR(100),  
    FOREIGN KEY (Emp_ID) REFERENCES Employee (Emp_ID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);  
GO
```

Потім наповнимо:

```
INSERT INTO Department VALUES (1, 'IT'), (2, 'HR');  
GO  
  
INSERT INTO Employee VALUES  
(101, 'Alice', 1),  
(102, 'Bob', 1),  
(201, 'Charlie', 2);  
GO  
  
INSERT INTO Equipment VALUES  
(1, 101, 'Laptop'),  
(2, 102, 'Tablet'),  
(3, 201, 'Monitor');  
GO
```

Results		Messages	
	Dept_ID	Dept_Name	
1	1	IT	
2	2	HR	
	Emp_ID	Name	Dept_ID
1	101	Alice	1
2	102	Bob	1
3	201	Charlie	2
	Equip_ID	Emp_ID	Equipment_Name
1	1	101	Laptop
2	2	102	Tablet
3	3	201	Monitor

Рисунок 4 - вигляд таблиць до операцій

Далі спробуємо виконати `DELETE FROM Department WHERE Dept_ID = 1`; Після цього, має зникнути айти відділ, а з ним і всі люди що працювали в ньому разом із технікою.

Results		Messages	
	Dept_ID	Dept_Name	
1	2	HR	
	Emp_ID	Name	Dept_ID
1	201	Charlie	2
	Equip_ID	Emp_ID	Equipment_Name
1	3	201	Monitor

Рисунок 5 - вигляд після каскадного видалення

Далі: `UPDATE Department SET Dept_ID = 101 WHERE Dept_ID = 2`;

Results

Messages

	Dept_ID	Dept_Name
1	1	IT
2	101	HR

	Emp_ID	Name	Dept_ID
1	101	Alice	1
2	102	Bob	1
3	201	Charlie	101

	Equip_ID	Emp_ID	Equipment_Name
1	1	101	Laptop
2	2	102	Tablet
3	3	201	Monitor

Рисунок 6 результат після каскадного андейта

Як бачимо, значення змінилося і в зв'язаних таблицях.

Завдання 6.

Тут просто треба вивести за сортуванням. Спробуємо різні варіанти на різних таблицях.

```

SELECT * FROM Worker
ORDER BY Salary ASC;

SELECT * FROM Farm
ORDER BY Area DESC;

SELECT * FROM Tech
ORDER BY Purchase_Date DESC;

SELECT * FROM Culture
ORDER BY Amount DESC;

SELECT * FROM [Transaction]
ORDER BY Money_transfer DESC;

SELECT * FROM Worker
ORDER BY Surname ASC, Name ASC;
```

Results Messages											
	Worker_ID	Brigade_ID	Fam_ID	Brigadir_ID	Name	Surname	Position	Rescue_Date	Salary	Status	Transaction_ID
1	51	51	51	30	Name51	Surname51	Worker	2023-05-25	5053.46	Active	80
2	11	11	11	65	Name11	Surname11	Tractor driver	2023-04-02	5141.72	Dismissed	96
3	8	8	8	94	Name8	Surname8	Worker	2024-01-12	5227.77	Dismissed	15
4	9	9	9	71	Name9	Surname9	Tractor driver	2024-12-26	5265.92	Dismissed	81
5	29	29	29	5	Name29	Surname29	Worker	2022-08-11	5349.78	Dismissed	57
6	3	3	3	NULL	Name3	Surname3	Foreman	2023-04-14	5416.00	Dismissed	66
7	99	99	99	4	Name99	Surname99	Tractor driver	2024-09-23	5517.65	Dismissed	65

	Fam_ID	Name	Area	Culture_ID
1	42	Fam 42	199	9
2	33	Fam 33	196	43
3	13	Fam 13	195	23
4	64	Fam 64	195	67
5	75	Fam 75	195	54
6	11	Fam 11	192	91
7	20	Fam 20	190	23
8	54	Fam 54	190	76

	Tech_ID	Brigade_ID	Fam_ID	Name	Type	Purchase_Date	Status	Transaction_ID
1	11	11	11	Combine	Type 3	2024-12-18	Broken	41
2	99	99	99	Tractor	Type 4	2024-11-25	UnderRepair	67
3	44	44	44	Combine	Type 5	2024-11-24	UnderRepair	73
4	59	59	59	Seeder	Type 1	2024-11-22	Working	39
5	96	96	96	Combine	Type 5	2024-11-13	UnderRepair	52
6	62	62	62	Tractor	Type 2	2024-11-03	UnderRepair	45
7	88	88	88	Plow	Type 2	2024-11-01	Working	16
8	69	69	69	Combine	Type 5	2024-10-26	Working	97

	Culture_ID	Name	Season	Amount	Fertilizers
1	49	Sunflower	Autumn	40	Nitrogen
2	75	Potato	Spring	40	Complex
3	98	Barley	Sum...	40	Organic
4	58	Sunflower	Sum...	39	Potassi...
5	67	Corn	Autumn	39	Potassi...
6	68	Sunflower	Sum...	39	Phosp...
7	20	Wheat	Autumn	39	Nitrogen
8	21	Barley	Sum...	39	Phosp...

	Transaction_ID	Transaction_Date	Money_transfer	Transaction_type
1	29	2024-04-10	49292.24	Sale
2	22	2024-01-30	48934.54	Repair
3	18	2023-02-20	48321.86	Purchase
4	39	2022-03-18	48040.83	Repair
5	25	2023-04-12	47740.09	Salary
6	100	2022-08-20	46637.96	Salary
7	49	2024-07-24	46120.20	Salary
8	71	2022-10-04	46009.71	Purchase

	Worker_ID	Brigade_ID	Fam_ID	Brigadir_ID	Name	Surname	Position	Rescue_Date	Salary	Status	Transaction_ID
1	1	1	1	53	Name1	Surname1	Worker	2023-03-22	12542.85	Active	15
2	10	10	10	5	Name10	Surname10	Tractor driver	2023-10-07	19277.26	Active	37
3	100	100	100	3	Name...	Surname...	Worker	2022-10-29	11701.93	Active	15
4	11	11	11	65	Name11	Surname11	Tractor driver	2023-04-02	5141.72	Dis...	96
5	12	12	12	7	Name12	Surname12	Tractor driver	2024-08-19	16111.29	Active	66
6	13	13	13	NULL	Name13	Surname13	Foreman	2023-04-11	7089.95	Dis...	73
7	14	14	14	71	Name14	Surname14	Tractor driver	2023-08-18	17799.32	Dis...	67
8	15	15	15	71	Name15	Surname15	Worker	2023-03-03	7105.23	Dis...	15

Рисунок 7 - відсортований вивід

Завдання 7.

```

SELECT * FROM Worker
WHERE Salary > 10000 AND Status = 'Active';

SELECT * FROM Farm
WHERE Area >= 150 OR Culture_ID < 3;

SELECT * FROM Tech
WHERE NOT Status = 'Working';

SELECT * FROM [Transaction]
WHERE Money_transfer <> 0;

```

```

SELECT * FROM Culture
WHERE Season = 'Spring' AND Amount <= 30;

SELECT * FROM Worker
WHERE Position = 'Tractor driver' OR Position = 'Agronomist';

```

Results Messages											
	Worker_ID	Brigade_ID	Fam_ID	Brigadir_ID	Name	Surname	Position	Rescue_Date	Salary	Status	Transaction_ID
1	1	1	1	53	Name1	Sumame1	Worker	2023-03-22	12542.85	Active	15
2	5	5	5	NULL	Name5	Sumame5	Foreman	2022-08-17	17069.79	Active	44
3	6	6	6	94	Name6	Sumame6	Worker	2022-10-07	13497.87	Active	52
4	10	10	10	5	Name10	Sumame10	Tractor driver	2023-10-07	19277.26	Active	37
5	12	12	12	7	Name12	Sumame12	Tractor driver	2024-08-19	16111.29	Active	66
6	16	16	16	92	Name16	Sumame16	Worker	2023-10-09	12319.07	Active	72
7	24	24	24	13	Name24	Sumame24	Tractor driver	2023-05-08	15834.21	Active	14
8	26	26	26	43	Name26	Sumame26	Worker	2023-08-02	17302.16	Active	80
	Fam_ID	Name	Area	Culture_ID							
1	7	Fam 7	71	2							
2	11	Fam 11	192	91							
3	13	Fam 13	195	23							
4	15	Fam 15	188	4							
5	17	Fam 17	169	92							
6	20	Fam 20	190	23							
7	23	Fam 23	180	42							
8	24	Fam 24	150	65							
	Tech_ID	Brigade_ID	Fam_ID	Name	Type	Purchase_Date	Status	Transaction_ID			
1	1	1	1	Combine	Type 4	2023-12-17	Broken	5			
2	2	2	2	Plow	Type 3	2024-10-14	UnderRepair	18			
3	4	4	4	Combine	Type 5	2022-09-14	Broken	69			
4	5	5	5	Seeder	Type 2	2022-08-14	Broken	41			
5	6	6	6	Plow	Type 4	2023-07-05	Broken	71			
6	7	7	7	Seeder	Type 1	2023-12-17	Broken	78			
7	8	8	8	Plow	Type 5	2022-07-10	Broken	8			
8	11	11	11	Combine	Type 3	2024-12-18	Broken	41			
	Transaction_ID	Transaction_Date	Money_transfer	Transaction_type							
1	1	2022-11-18	16811.63	Salary							
2	2	2022-11-04	39883.07	Salary							
3	3	2023-05-08	35691.88	Salary							
4	4	2024-10-14	30451.71	Salary							
5	5	2023-06-20	43028.27	Sale							
6	6	2022-04-18	4538.70	Salary							
7	7	2023-01-10	43543.71	Salary							
8	8	2024-04-15	9070.92	Salary							
	Culture_ID	Name	Season	Amount	Fertilizers						
1	3	Wheat	Spring	27	Organic						
2	5	Barley	Spring	20	Potassium						
3	6	Barley	Spring	22	Potassium						
4	9	Sunflower	Spring	22	Complex						
5	10	Barley	Spring	25	Organic						
6	13	Barley	Spring	28	Phosph...						
7	27	Barley	Spring	25	Phosph...						
8	37	Barley	Spring	25	Complex						
	Worker_ID	Brigade_ID	Fam_ID	Brigadir_ID	Name	Surname	Position	Rescue_Date	Salary	Status	Transaction_ID
1	9	9	9	71	Name9	Sumame9	Tractor driver	2024-12-26	5265.92	Dismissed	81
2	10	10	10	5	Name10	Sumame10	Tractor driver	2023-10-07	19277.26	Active	37
3	11	11	11	65	Name11	Sumame11	Tractor driver	2023-04-02	5141.72	Dismissed	96
4	12	12	12	7	Name12	Sumame12	Tractor driver	2024-08-19	16111.29	Active	66
5	14	14	14	71	Name14	Sumame14	Tractor driver	2023-08-18	17799.32	Dismissed	67
6	20	20	20	33	Name20	Sumame20	Tractor driver	2023-04-14	6927.15	Active	32
7	24	24	24	13	Name24	Sumame24	Tractor driver	2023-05-08	15834.21	Active	14
8	25	25	25	43	Name25	Sumame25	Tractor driver	2023-01-30	6982.80	Active	50
9	21	21	21	55	Name21	Sumame21	Tractor driver	2023-02-21	7308.08	Dismissed	77

Рисунок 8 - результат виводу більше менше равно

Завдання 8.

Насправді про JOIN і типи було в минулій роботі, тому було б доречно просто скопіювати приклад звідти, але оскільки я перейшов на MS SQL, то покажу нову версію того прикладу. В файлі QUERY.SQL вони знаходяться по коментарю --20:

```
SELECT w.[Name], w.[Surname], tr.[Transaction_type], tr.[Money_transfer]
FROM [Worker] AS w
JOIN [Transaction] AS tr
  ON w.[Transaction_ID] = tr.[Transaction_ID];

SELECT f.[Name] AS [FarmName], c.[Name] AS [CultureName]
FROM [Farm] AS f
LEFT JOIN [Culture] AS c
  ON f.[Culture_ID] = c.[Culture_ID];

SELECT w.[Name], w.[Surname], tr.[Transaction_type]
FROM [Worker] AS w
LEFT JOIN [Transaction] AS tr
  ON w.[Transaction_ID] = tr.[Transaction_ID];

SELECT t.[Name] AS [TechName], b.[Name] AS [BrigadeName]
FROM [Brigade] AS b
RIGHT JOIN [Tech] AS t
  ON t.[Brigade_ID] = b.[Brigade_ID]
 AND t.[Farm_ID] = b.[Farm_ID];

SELECT b.[Name] AS [BrigadeName], w.[Name] AS [WorkerName]
FROM [Brigade] AS b
LEFT JOIN [Worker] AS w
  ON w.[Brigade_ID] = b.[Brigade_ID]
 AND w.[Farm_ID] = b.[Farm_ID];

SELECT c.[Name] AS [CultureName], f.[Name] AS [FarmName]
FROM [Culture] AS c
RIGHT JOIN [Farm] AS f
  ON f.[Culture_ID] = c.[Culture_ID];
```

Results					Messages				
	Name	Surname	Transaction_type	Money_transfer					
1	Name1	Sumame1	Salary	1182.74					
2	Name2	Sumame2	Salary	40705.02					
3	Name3	Sumame3	Salary	11632.00					
4	Name4	Sumame4	Sale	17623.64					
5	Name5	Sumame5	Salary	9091.26					
6	Name6	Sumame6	Salary	15851.73					
7	Name7	Sumame7	Purchase	28523.17					
8	Name8	Sumame8	Salary	1182.74					

	FamName	CultureName			
7	Fam 7	Barley			
8	Fam 8	Barley			
9	Fam 9	Barley			
10	Fam 10	Sunflower			
11	Fam 11	Wheat			
12	Fam 12	Sunflower			
13	Fam 13	Sunflower			
14	Fam 14	Corn			

	Name	Surname	Transaction_type		
1	Name1	Sumame1	Salary		
2	Name2	Sumame2	Salary		
3	Name3	Sumame3	Salary		
4	Name4	Sumame4	Sale		
5	Name5	Sumame5	Salary		
6	Name6	Sumame6	Salary		
7	Name7	Sumame7	Purchase		
8	Name8	Sumame8	Salary		

	TechName	BrigadeName			
1	Combine	Brigade 1			
2	Plow	Brigade 2			
3	Tractor	Brigade 3			
4	Combine	Brigade 4			
5	Seeder	Brigade 5			
6	Plow	Brigade 6			
7	Seeder	Brigade 7			
8	Plow	Brigade 8			

	BrigadeName	WorkerName			
1	Brigade 1	Name1			
2	Brigade 2	Name2			
3	Brigade 3	Name3			
4	Brigade 4	Name4			
5	Brigade 5	Name5			
6	Brigade 6	Name6			
7	Brigade 7	Name7			
8	Brigade 8	Name8			

	CultureName	FamName			
1	Barley	Fam 1			
2	Corn	Fam 2			
3	Wheat	Fam 3			
4	Wheat	Fam 4			
5	Potato	Fam 5			
6	Sunflower	Fam 6			
7	Barley	Fam 7			
8	Barley	Fam 8			

Рисунок 9 - скріншот результата джоїнов

Завдання 9.

Це теж було в минулій лабороторній, тому просто перероблюю під MS SQL, в файлі --35:

```

SELECT c.[Fertilizers], COUNT(f.[Farm_ID]) AS [FarmCount]
FROM [Culture] AS c
JOIN [Farm] AS f
  ON c.[Culture_ID] = f.[Culture_ID]
GROUP BY c.[Fertilizers]
HAVING COUNT(f.[Farm_ID]) > 2;

-- 36
SELECT c.[Season], COUNT(f.[Farm_ID]) AS [FarmCount]
FROM [Culture] AS c
JOIN [Farm] AS f
  ON f.[Culture_ID] = c.[Culture_ID]
GROUP BY c.[Season]
HAVING COUNT(f.[Farm_ID]) >= 1;

-- 37
SELECT w.[Name], w.[Surname], w.[Salary]
FROM [Worker] AS w
JOIN [Brigade] AS b
  ON w.[Brigade_ID] = b.[Brigade_ID]
  AND w.[Farm_ID] = b.[Farm_ID]
WHERE w.[Salary] > (
  SELECT AVG([Salary])
  FROM [Worker]
);

-- 38
SELECT t.[Name] AS [TechName], t.[Status], tr.[Money_transfer]
FROM [Tech] AS t
JOIN [Transaction] AS tr
  ON t.[Transaction_ID] = tr.[Transaction_ID]
WHERE tr.[Money_transfer] > 20000;

-- 39
SELECT f.[Name] AS [FarmName], AVG(w.[Salary]) AS [AvgSalary]
FROM [Farm] AS f
JOIN [Brigade] AS b
  ON f.[Farm_ID] = b.[Farm_ID]
JOIN [Worker] AS w
  ON w.[Brigade_ID] = b.[Brigade_ID]
  AND w.[Farm_ID] = b.[Farm_ID]
GROUP BY f.[Farm_ID], f.[Name]
HAVING AVG(w.[Salary]) > 10000;

```



```
-- 40
SELECT c.[Name] AS [CultureName], COUNT(f.[Farm_ID]) AS [FarmCount]
FROM [Culture] AS c
JOIN [Farm] AS f
  ON f.[Culture_ID] = c.[Culture_ID]
WHERE f.[Area] > 100
GROUP BY c.[Culture_ID], c.[Name]
HAVING COUNT(f.[Farm_ID]) >= 2;
```

Results		Messages	
	Fertilizers	FamCount	
1	Complex	35	
2	Nitrogen	9	
3	Organic	24	
4	Phosphorus	15	
5	Potassium	17	
	Season	FamCount	
1	Autumn	38	
2	Spring	26	
3	Summer	36	
	Name	Surname	Salary
1	Name1	Sumame1	12542.85
2	Name4	Sumame4	13896.34
3	Name5	Sumame5	17069.79
4	Name6	Sumame6	13497.87
5	Nam...	Sumam...	19277.26
6	Nam...	Sumam...	16111.29
7	Nam...	Sumam...	17799.32
8	Nam...	Sumam...	12319.07
	TechName	Status	Money_transfer
1	Combine	Broken	43028.27
2	Plow	UnderRepair	48321.86
3	Plow	Broken	46009.71
4	Seeder	Broken	32559.11
5	Plow	Broken	28523.17
6	Tractor	Working	31614.46
7	Seeder	Working	44272.86
8	Tractor	Working	20433.15
	FamName	AvgSalary	
1	Fam 1	12542.850000	
2	Fam 4	13896.340000	
3	Fam 5	17069.790000	
4	Fam 6	13497.870000	
5	Fam 7	11923.010000	
6	Fam 10	19277.260000	
7	Fam 12	16111.290000	
8	Fam 14	17799.320000	
	CultureName	FamCount	
1	Wheat	3	
2	Barley	2	
3	Sunflower	3	
4	Potato	2	

Рисунок 10 - скріншот виконання запитів з групами

Завдання 10.

Оскільки в мене справді багато запитів, то буде складно перевірити і проаналізувати всі, тому я обрав невеличку вибірку з тих, що проаналізую. Спробуємо проаналізувати перший сьогоднішній запит:

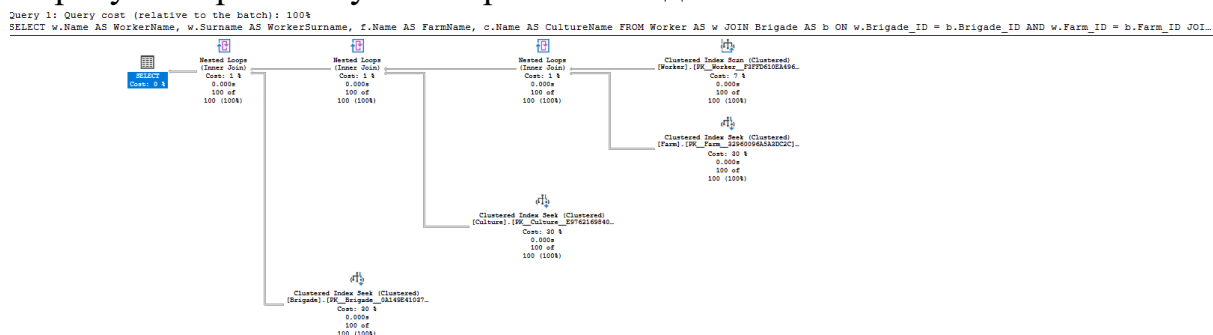


Рисунок 11 - схема аналізу першого запиту

JOIN Worker -> Brigade

Відбувається через Clustered Index Seek, що означає — SQL Server швидко знаходить потрібні рядки по індексу FK_Brigade.

Операція: Nested Loops (Inner Join) — добра для невеликих обсягів або коли індекси є.

JOIN Brigade -> Farm

Знову Clustered Index Seek — ефективний доступ по зовнішньому ключу Farm_ID.

JOIN Farm -> Culture

Знову використовується Clustered Index Seek — SQL Server використовує первинний ключ Culture_ID.

Scan у Worker

Clustered Index Scan (а не Seek) означає, що сервер читає всю таблицю Worker, бо можливо, не може використати ефективний індекс для Brigade_ID + Farm_ID

Другим спробуємо цей:

```
SELECT * FROM Worker
WHERE Salary > 10000 AND Status = 'Active';
```

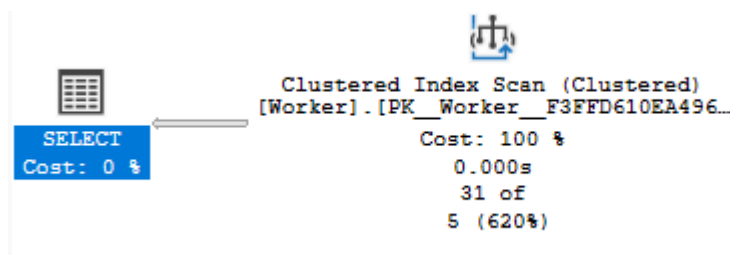


Рисунок 12 - скріпшот аналізу 2

Clustered Index Scan — означає, що SQL Server сканує всю таблицю, читаючи всі рядки з кластеризованого індексу (PRIMARY KEY).

Cost: 100% — цей запит був єдиний у партії, тому всі ресурси (витрати) віднесені до нього.

31 of 5 (620%) — фактично оброблено 31 рядок, хоча планував лише 5. Це означає помилкову оцінку кількості рядків (неактуальна статистика).

Третім візьмемо останній наш запит в цій роботі.

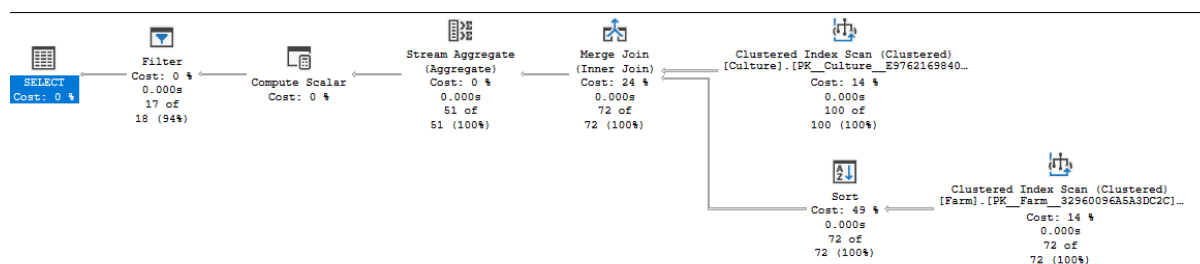


Рисунок 13 - скріпшот аналізу 3

Clustered Index Scan (Culture) – зчитує всі записи з таблиці Culture через первинний ключ. Clustered Index Scan (Farm) – аналогічно, читає всі ферми. Sort – потрібен, щоб підготувати дані до Merge Join. Merge Join (Inner Join) – об'єднує Culture і Farm за ключем. Stream Aggregate – обчислює AVG(f.Area). Compute Scalar – розраховує значення агрегатів. Filter – фільтрує записи, де AVG(f.Area) > загального середнього. SELECT – повертає результат користувачу. Запит виконує об'єднання і агрегацію, тому потребує сортування, тож це може бути дорогим по ресурсам.

Висновок:

У ході виконання лабораторної роботи було створено базу даних відповідно до індивідуального варіанта, реалізовано структуру таблиць з первинними та зовнішніми ключами, включаючи обмеження з каскадним оновленням та видаленням. Було побудовано численні SQL-запити з використанням ORDER BY, булевих та реляційних операторів, різних типів JOIN, а також GROUP BY та HAVING для статистичного аналізу. Для деяких запитів було виконано аналіз продуктивності з використанням Execution Plan.

GitHub з файлами практичної: <https://github.com/ddanq4/3-obd/tree/main/2%20semeestr/lab3>